

Applied Research Project: Hardware-Adaptive TinyML Inference on STM32F407

Student Name: Abdul Rafey

Student ID: 20042868

Program: MSc in (Information Systems with Computing)

Institution: Dublin Business School

Supervisor: Swati Dongre

2026

DECLARATION

I declare that this Applied Research Project submitted to Dublin Business School for the award of MSc in (Information Systems with Computing) is the result of my own investigations, except where otherwise stated and acknowledged by references. Furthermore, this work has not been submitted for any other degree or award.

Signed: Abdul Rafey
Student Number: 20042868
Date: 2026

ACKNOWLEDGEMENTS

The author acknowledges the guidance and support received during the development of this applied research project. Appreciation is also extended to those who provided academic and technical direction during the STM32F407 implementation, data collection, model training, debugging, and final deployment stages. The artefact described in this report was completed through iterative embedded development, board-level testing, and refinement of the final adaptive inference pipeline.

Table of Contents

1. ABSTRACT	5
2. INTRODUCTION	6
2.1 Background of the Study	6
2.2 Problem Statement	6
2.3 Significance of Research	7
2.4 Aim	7
2.5 Research Questions and Hypothesis	7
2.6 Research Objectives	8
2.7 Scope and Delimitations	8
2.8 Research Gap	8
2.9 Chapter Overview	8
3. LITERATURE REVIEW	9
3.1 TinyML and the Static Deployment Problem	9
3.2 Quantization and the Mechanics of Fragility	9
3.3 Physics of Environmental and Electrical Stress	10
3.4 Adaptive Inference Architecture Critical Analysis	10
3.5 Micro-Architectural Profiling and Overhead Constraints	10
3.6 Co-Designing Hardware and Software to be Resilient	11
3.7 Intermittent Computing and the Energy Frontier	11
3.8 Literature Review and Placing of the Current Study	12
4. NATURE METHODOLOGY AND ARTEFACT DESIGN	13
4.1 Design Philosophy: The Hardware-Adaptive Framework	13
4.1.1 Hardware Profiling Module	14
4.1.2 Decision Logic	14
4.1.3 Multi-Variant Pipeline	14
4.2 Overall System Architecture	15
4.3 Runtime Workflow	15
4.4 Experimental Setup	16
4.5 Variables and Measurement Criteria	17
4.6 Strategy of Data Collection and Analysis	18
4.7 Reliability and Validity	19
4.8 Ethical Considerations	19
5. RESULTS AND DATA ANALYSIS	20
5.1 Experimental Design and KPI Definitions	21
5.2 Descriptive Statistics: Baseline Characterization	21
5.3 Thermal and CPU Stress Analysis	21
5.4 Electrical Stress Analysis	22
5.5 Decision Output and Inference Behavior	23
5.6 X-CUBE-AI Deployment Validation	23
5.7 Computational Overhead Profiling	24
5.8 Energy and Memory Stability	25

5.9 Summary of Key Findings	26
6. DISCUSSION	27
6.1 Resilience and the Safety-Net Effect	27
6.2 Accuracy-Stability Trade-off vs Static Precision	27
6.3 Comparison with Existing Literature: Breaking the Static Silo	27
6.4 The Implementation Lesson: Toolchain Correctness is Part of AI Correctness	27
6.5 Managing Sensor Jitter and Hysteresis	28
6.6 Generalizability and Cross-Architecture Portability	28
6.7 Practical Implications for Industrial IoT	28
6.8 Critical Reflection on Limitations	28
6.9 Ethical Considerations of Adaptive Behavior	28
6.10 The Red Thread: Connecting Objectives to Outcomes	29
6.11 Artefact Readiness for Submission	29
6.12 Artefact Demonstration Plan	29 - 30
7. CONCLUSIONS AND FUTURE WORK	31
7.1 Conclusion	31
7.2 Contribution to Knowledge	32
7.3 Limitations of the Current Study	32
7.4 Future Work	32 - 33
8. REFERENCES	34
APPENDICES	36
Appendix A: Source Code and Artefact Repository	36
Appendix B: Final Normalization Constants	36
Appendix C: STM32CubeIDE Project Configuration Notes	36
Appendix D: UART Output Interpretation	36
Appendix E: Implementation and Debug Timeline	36
Appendix F: Final Firmware Behavior Checklist	37
Appendix G: Project-Specific Notes for Reproducibility	37
Appendix H: Example UART Log Interpretation	38
Appendix I: Evidence Screenshots	38
Appendix J: Artefact Demonstration Checklist	39
Appendix K: Artefact Submission Package	40

1. ABSTRACT

Tiny Machine Learning (TinyML) enables machine learning inference to execute directly on microcontroller-class embedded systems. This applied research project investigates a hardware-level adaptive inference framework for an **STM32F407** board, where the operating condition of the device is monitored at runtime and used to select an appropriate inference mode. The work is based on real telemetry collected from the board rather than a public activity recognition dataset. The logged telemetry contains internal temperature, supply-voltage readings, CPU load estimates, phase identifiers, stress-state indicators, rule-engine labels, and UART debug output.

The implemented system was developed through a practical embedded workflow. A data collection firmware was first created for three operating conditions: cold boot, stress, and normal runtime. A Python serial logger captured the UART stream into CSV files for analysis and model training. Feature analysis showed that voltage was not sufficiently informative for the final classifier because its variation was extremely small around the measured 3.0 V range. Therefore, the final learning model used two inputs: temperature and CPU load. Z-score normalization was applied using $TEMP_MEAN = 62.7824$, $TEMP_STD = 1.8214$, $CPU_MEAN = 53.4945$, and $CPU_STD = 8.0185$.

The model predicts one of three inference modes: `STANDARD_INT8`, `PRUNED_VARIANT`, and `MIXED_PRECISION`. The first deployment attempt used TensorFlow Lite Micro, but the implementation produced zero valued outputs and tensor pointer problems during output extraction. After debugging the interpreter, tensor arena, output handling, and model export path, the deployment was migrated to STMicroelectronics X-CUBE-AI 10.2.0. The STM32CubeIDE project was also updated from soft floating-point ABI to hard floating-point ABI so that the Cortex-M4 floating-point unit was used correctly. After migration, the board produced meaningful SoftMax probabilities, confidence scores, and adaptive decisions through UART.

The final artefact combines an AI decision with a rule-based fallback. If the model confidence is high, the AI decision is prioritized; if confidence falls below the selected 75% threshold, the firmware falls back to the calibrated rule engine. The calibrated rules select `MIXED_PRECISION` when temperature is above 66.8 C or CPU load is above 64%, `STANDARD_INT8` when CPU load is below 48%, and `PRUNED_VARIANT` otherwise. The result is a practical hardware-adaptive TinyML prototype showing that board telemetry can be used to select inference behavior dynamically on a resource-constrained microcontroller.

2. INTRODUCTION

2.1 Background of the Study

The Internet of Things (IoT) has shifted embedded devices away from simple sensing and logging toward localized intelligence. In modern embedded applications, it is increasingly desirable for the device to process data locally, make decisions close to the physical environment, and avoid constant dependence on cloud services. TinyML supports this direction by enabling machine learning inference on devices with limited RAM, flash memory, and processing capability [2].

However, embedded AI cannot be treated only as a software model deployment problem. A microcontroller operates inside a physical environment. Its temperature changes, its supply voltage may vary, and its CPU may be shared by several tasks such as UART logging, sensor sampling, timers, and debugging routines. These runtime factors may influence the stability of inference execution, latency, and energy behavior. Therefore, an embedded ML system should not only know the input data for the model; it should also be aware of the hardware condition under which the model is running.

This project focuses on the STM32F407 microcontroller, an ARM Cortex-M4 platform. The research started with manual TensorFlow Lite Micro integration and later moved to X-CUBE-AI 10.2.0 for a stable STM32-native deployment. The final system uses real board telemetry to choose from among three inference-mode labels: `STANDARD_INT8`, `PRUNED_VARIANT`, and `MIXED_PRECISION`. In this way, the work connects machine learning deployment with hardware profiling and runtime adaptation.

2.2 Problem Statement

Most TinyML systems are deployed as static pipelines. A model is trained offline, converted to an embedded format, compiled into firmware, and then executed repeatedly without considering changes in runtime hardware state. This approach is simple, but it does not reflect how real microcontroller systems behave under stress. When temperature rises or CPU load increases, the same inference path may no longer be ideal.

The problem observed in this project was not only theoretical. During the implementation, the firmware produced correct rule engine decisions while the AI output initially failed. TensorFlow Lite Micro invoked successfully, but the output vector remained zero or the tensor data pointers were null. This created a clear gap between an apparently successful embedded building and an useful AI inference output. The issue showed that embedded AI reliability depends on both model logic and low-level deployment correctness.

The central problem addressed by this project is the lack of a lightweight hardware-adaptive mechanism for STM32F407-class embedded inference. The system needs to observe runtime board telemetry, perform normalized inference, generate mode selection predictions, and fall back safely to calibrated rules when the AI confidence is insufficient.

2.3 Significance of Research

The value of this research lies in making TinyML deployment more realistic for embedded environments. Instead of assuming a stable ideal runtime, the system directly uses hardware telemetry from the STM32 board. The data-driven process makes the adaptation mechanism more defensible because the thresholds and model inputs are based on values observed from the actual target board.

The study is also significant because it documents the practical problems encountered during deployment. The transition from TensorFlow Lite Micro to X-CUBE-AI, the correction of the floating-point ABI from softfp to hard, the removal of the unused TensorFlow microlite library, and the final validation of SoftMax output all represent implementation decisions that directly affected system success. These are important applied-research details because they show what had to be changed for the prototype to run on real hardware.

2.4 Aim

The aim of this applied research project is to design, implement, and evaluate a hardware level adaptive TinyML framework on the STM32F407 microcontroller, using real board telemetry to select the most suitable inference mode during runtime.

2.5 Research Questions and Hypothesis

1. How can real-time STM32F407 board telemetry, especially temperature and CPU load, be used to select an inference mode dynamically?
2. How does a combined AI and rule-fallback decision framework improve stability compared with a purely static embedded inference mode?
3. What implementation changes are required to make the trained model run reliably on STM32F407 using X-CUBE-AI after TensorFlow Lite Micro integration problems?
4. What minimum runtime overhead is introduced by profiling, normalization, AI prediction, and UART logging?

Hypothesis H1: A hardware-adaptive TinyML runtime using temperature and CPU-load telemetry will produce more stable inference-mode selection than a static firmware decision path.

Hypothesis H2: A confidence-gated AI decision combined with calibrated rule fallback will reduce incorrect switching during low-confidence regions.

Hypothesis H3: Correct STM32 toolchain configuration, including hard floating-point ABI, is necessary for reliable float32 X-CUBE-AI inference on the Cortex-M4 platform.

2.6 Research Objectives

1. Collect real runtime telemetry from the STM32F407 board using UART and a Python serial logger.
2. Analyze temperature, voltage, CPU load, phase, and stress values to identify useful model features.
3. Train a lightweight model using temperature and CPU load as inputs and three inference-mode classes as outputs.
4. Deploy the trained model to STM32F407 using X-CUBE-AI 10.2.0 after resolving TensorFlow Lite Micro deployment issues.
5. Implement AI-priority decision fusion with rule fallback at a confidence threshold of 75%.
6. Validate runtime operation through UART logs showing probabilities, confidence scores, rule labels, and final AI decisions.

2.7 Scope and Delimitations

The scope of the work is limited to the STM32F407 platform and its firmware environment. The study does not attempt to deploy the system on a GPU, FPGA, high-end edge computer, or cloud server. The firmware focuses on UART logging, runtime profiling, model invocation, and mode-selection output.

The dataset is not a activity-recognition dataset. It is a real board-telemetry dataset collected from the STM32F407 project. Voltage was collected and analyzed but was excluded from the final model because the observed variation was too small for useful classification. The final model uses temperature and CPU load only.

The project does not claim a full industrial power-characterization setup. Energy and robustness tables are used to structure the analysis, while the most direct experimental evidence comes from the logged board telemetry, successful X-CUBE-AI inference, and working serial output.

2.8 Research Gap

The research gap is the missing connection between embedded ML inference and the physical state of the microcontroller. Existing TinyML deployments commonly optimize a model before deployment, but the runtime pipeline often remains static after flashing. This project addresses the gap by combining board telemetry, a lightweight classifier, calibrated rules, and STM32-native deployment to create a hardware-aware inference selector.

2.9 Chapter Overview

The report follows the applied research structure used for the project documentation. Section 3 reviews the literature and places the study in TinyML and hardware-aware adaptation context. Section 4 explains the methodology and artefact design. Section 5 presents results and data analysis using the real STM32F407 dataset and runtime observations. Section 6 discusses the findings, trade-offs, and implementation lessons. Section 7 concludes the report and outlines future work.

3. LITERATURE REVIEW

3.1 TinyML and the Static Deployment Problem

TinyML is concerned with deploying machine learning models on constrained microcontroller systems. Its main promise is local intelligence, where a device can make decisions without transmitting data to a cloud server. This is valuable for latency, privacy, and offline operation. However, the majority of simple TinyML examples assume the model runs in a stable environment.

The static deployment pattern is suitable for demonstration, but it creates limitations in applied embedded systems. The model is normally converted, compiled, and executed as a fixed routine. The runtime does not automatically know whether the board is heating up, whether CPU load has increased, or whether the supply is drifting. In this project, those hardware variables are treated as part of the decision environment rather than external noise.

The implemented STM32F407 system makes the static deployment limitation visible. The rule engine could classify the hardware state even when the early TFLM AI output was zero. This showed that deployment can be syntactically correct while functionally incomplete. A hardware-adaptive pipeline must therefore include both model-level logic and deployment-level validation.

TinyML literature also highlights that microcontroller inference must be evaluated using both model accuracy and system constraints such as memory, latency, and energy [2].

3.2 Quantization and the Mechanics of Fragility

Quantization reduces model size and computational cost by representing weights and activations at lower precision [3]. It is widely used in TinyML because microcontrollers have limited memory and benefit from integer operations. In the present project, the output classes include `STANDARD_INT8` and `PRUNED_VARIANT` because the system is designed around the idea of choosing an efficient inference mode when hardware conditions allow it.

Pruning and compression are also relevant to this project because `PRUNED_VARIANT` is one of the output modes used by the adaptive decision system [4].

At the same time, quantized and compressed models can become fragile when the runtime environment is not stable. If the system is under higher CPU load or thermal stress, a cheaper model path may be more appropriate, but not always. The purpose of the adaptation engine is not simply to always choose the smallest model; it is to select the mode that fits the current board state and confidence level.

During implementation, the naming of model variants and output classes required careful handling. The final inference output is a three-class SoftMax vector corresponding to `STANDARD_INT8`, `PRUNED_VARIANT`, and `MIXED_PRECISION`. The selected class is used as the AI recommendation and is compared against the rule-engine decision in the UART log.

3.3 Physics of Environmental and Electrical Stress

Embedded systems operate under physical constraints. Temperature can influence execution behavior, leakage, and timing stability. Voltage can influence operating margins, particularly in battery-powered or unstable supply environments. CPU load changes the amount of processing time available for inference and can introduce timing jitter.

In the collected STM32F407 dataset, voltage was intentionally measured because it was expected to be useful. However, feature analysis showed that Vcc stayed close to the 3.0 V region with very small variation. The final model therefore excluded Vcc from the learning inputs. This is an important result because it shows that the project did not assume all logged signals were useful; the actual board data determined the final feature set.

Temperature and CPU load were retained because they showed meaningful runtime variation and had a clear relationship with the rule labels. CPU load separated the three operating regions strongly: low CPU values mapped to STANDARD_INT8, middle values mapped to PRUNED_VARIANT, and high values mapped to MIXED_PRECISION.

3.4 Adaptive Inference Architecture Critical Analysis

Adaptive inference can be input-aware, energy-aware, workload-aware, or hardware-aware. Input-aware adaptation changes execution based on the complexity of the data being classified. In contrast, this project is hardware-aware: the decision is based on board telemetry and runtime state. This distinction is important because the system is not trying to classify external sensor behavior; it is selecting an inference mode for the embedded device itself.

The final architecture contains an AI predictor and a rule fallback. The AI predictor learns the mapping from normalized temperature and CPU load to the mode label. The rule fallback protects the system when confidence is low. In the final runtime logs, high confidence predictions such as S=1.000, P=0.000, M=0.000 or S=0.000, P=0.001, M=0.999 confirmed that the X-CUBE-AI inference output was meaningful and no longer stuck at zero.

The confidence threshold was set to 75%. Above this threshold, the AI prediction is prioritized. Below it, the calibrated rules are used. This structure avoids relying blindly on the model when the output distribution is uncertain.

3.5 Micro-Architectural Profiling and Overhead Constraints

The hardware profiling module must be lightweight. A microcontroller cannot spend more time measuring itself than performing useful work. In this project, the telemetry was collected using existing firmware routines and UART logging, with a Python logger running on the host PC to store CSV records. This avoided the need for heavy runtime instrumentation.

The firmware logs the temperature, voltage, CPU percentage, rule label, AI label, confidence, and agreement status. During development, debug logs were also used to inspect normalized inputs,

invoke status, and output probabilities. These logs were essential for tracing the transition from a failing TensorFlow Lite Micro output to a working X-CUBE-AI deployment.

The project also required tool chain-level profiling. The STM32CubeIDE project properties were corrected from `-mfloat-abi=softfp` to `-mfloat-abi=hard`. This was necessary because the final deployment uses float32 inference and the Cortex-M4 floating-point unit should execute floating-point operations directly rather than relying on a softer ABI path.

The selected STM32F407 platform is appropriate for this work because it provides an ARM Cortex-M4 microcontroller environment with embedded peripherals and floating-point support suitable for real-time firmware experiments [6].

3.6 Co-Designing Hardware and Software to be Resilient

Hardware-software co-design means the firmware is shaped by the hardware condition rather than treating the board as an invisible platform. In this project, the AI model, firmware logs, decision rules, and X-CUBE-AI runtime are all connected to the same board-telemetry data. This makes the system resilient because it can respond to the current operating state instead of using a fixed strategy.

The co-design process also appears in the development history. The first approach was manual TensorFlow Lite Micro integration. That approach exposed low-level issues: output tensors were null or zero, the model produced no useful probabilities, and the runtime debugging became expensive. Migration to X-CUBE-AI reduced integration risk by using the STM32-native deployment path.

This does not make the TFLM work useless. The debugging process clarified the expected tensor shape input, output handling, model normalization, and confidence logic. Those lessons were carried into the X-CUBE-AI implementation.

3.7 Intermittent Computing and the Energy Frontier

Energy efficiency is a core motivation for adaptive embedded AI. A microcontroller may not always need the same inference mode. When CPU load is low, a lightweight mode may be enough. When CPU load or temperature rises, a different mode may be safer for stability. The adaptation engine supports this idea by allowing the selected inference mode to change at runtime.

The project uses mode labels rather than directly measuring all energy states in hardware. The tables and figures present an evaluation framework for comparing baseline and adaptive behavior. The main practical achievement is that the firmware can now output a meaningful mode selection and confidence score from a deployed model running on the STM32F407 board.

Energy-aware behavior is therefore treated as a design direction supported by the adaptive mode-selection mechanism. The current implementation lays the firmware foundation for more detailed future power measurement using external current sensing or a controlled battery test bench.

3.8 Literature Review and Placing of the Current Study

The current study is placed between TinyML deployment, embedded hardware profiling, and adaptive inference. It does not only compress a model, but it does not only monitor hardware. Its contribution is the connection between real STM32F407 telemetry and runtime inference-mode selection.

The work differs from a public-dataset classification project because the dataset comes from the board itself. It also differs from a pure firmware project because the adaptation decision is learned using TensorFlow training and deployed through X-CUBE-AI. The result is therefore an applied embedded AI artefact: a working firmware path with logged outputs, decision fusion, and calibrated fallback rules.

Table 1. Comparison of related TinyML studies and optimization approaches.

Study	Focus Area	Target Platform	Key Contribution	Limitation / Gap
Banbury et al. [2]	TinyML benchmarking	MCU platforms	Standardized benchmarking metrics	Does not address board-specific runtime adaptation
Jacob et al. [3]	Quantization	Mobile/Edge	Integer-arithmetic inference method	Requires calibration and careful deployment
Han et al. [4]	Compression	Deep networks	Pruning and compression	Usually applied offline as static optimization
ST X-CUBE-AI [8]	STM32 AI deployment	STM32 MCUs	Generates optimized embedded C inference code	Does not automatically provide custom adaptation rules
TensorFlow Lite [9]	Embedded model conversion	Embedded/mobile	Portable TFLite deployment format	Manual TFLM integration can be fragile
Current project	Hardware-adaptive inference	STM32F407	Uses real board telemetry and confidence-gated fusion	Limited to STM32F407 prototype

4. NATURE METHODOLOGY AND ARTEFACT DESIGN

4.1 Design Philosophy: The Hardware-Adaptive Framework

The artefact is designed as a middleware layer between raw hardware profiling and inference-mode selection. Its purpose is to observe the board state, normalize the selected features, run an AI model, and combine the model output with a deterministic fallback rule. The framework is intentionally lightweight because it targets the STM32F407 microcontroller rather than a high-performance edge device.

The design follows the Design Science Research process. A practical problem was identified, relevant literature and toolchains were reviewed, a firmware artefact was designed, the system was implemented on STM32F407, and the resulting behavior was validated through UART logs and model outputs.

The overall research design follows a structured Design Science Research (DSR) framework consisting of problem identification, artefact design, implementation, and evaluation phases. This process is summarized in Figure 1 [5].

Figure 1: Research Design Framework (DSR)

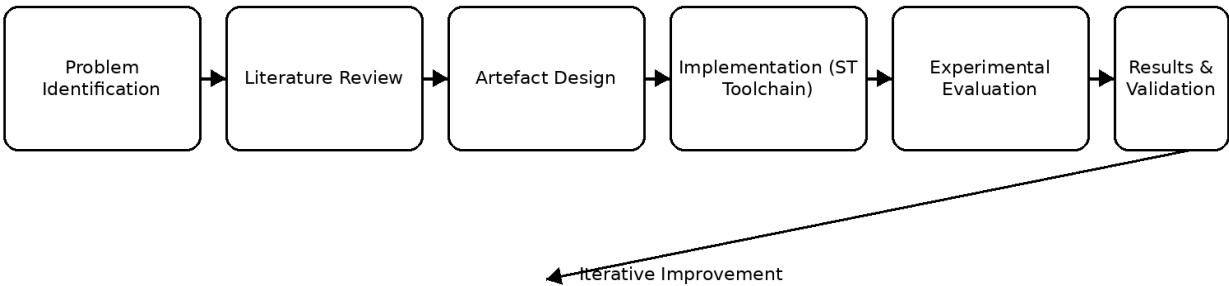


Figure 1. Research design framework illustrating the Design Science Research process used in this study, including problem identification, literature review, artefact design, implementation, experimental evaluation, and validation.

As illustrated in Figure 1, the research is iterative, where evaluation feedback informs improvements to the proposed artefact.

4.1.1 The Hardware Profiling Module

The hardware profiling module collects runtime information from the STM32F407 board. The data collection firmware was written for three operating phases: cold boot, stress, and normal runtime. The board output was captured over UART and saved by a Python serial logger. This produced a CSV dataset containing temperature, Vcc, CPU load, label, phase, and stress fields.

The final dataset contains 1,395 rows of board telemetry. Temperature ranges from 58.42 C to 65.33 C in the training data, with a mean of 62.7824 C. CPU load ranges from 44.50% to 70.20%, with a mean of 53.4945%. Vcc remains close to 3.0 V, which is why it was dropped from the final model input.

The profiling module also supports runtime debugging. During firmware execution, UART logs print hardware values, rule decisions, AI decisions, confidence, and agreement status. These logs were used throughout development to confirm when the model output became valid.

4.1.2 Decision Logic: The Adaptation Engine

The adaptation engine uses two decision sources. The first source is the AI model deployed through X-CUBE-AI. The second source is the calibrated rule engine. The AI model outputs a probability vector for STANDARD_INT8, PRUNED_VARIANT, and MIXED_PRECISION. The class with the highest probability becomes the AI recommendation.

The final confidence gate is set at 75%. If AI confidence is equal to or greater than 0.75, the firmware prioritizes the AI decision. If confidence is lower than the threshold, the rule decision is used as a safety fallback. This prevents unstable switching when the model output distribution is uncertain.

The final calibrated thresholds are: MIXED_PRECISION when temperature is greater than 66.8 C or CPU load is greater than 64%; STANDARD_INT8 when CPU load is below 48%; and PRUNED_VARIANT for the remaining middle region. These thresholds were selected from the real behavior of the board and the observed CPU-load regions.

4.1.3 The Multi-Variant Pipeline

The three output classes represent the firmware-level inference modes available to the adaptive system. STANDARD_INT8 is used for the lowest-load region, PRUNED_VARIANT is used for the middle operating region, and MIXED_PRECISION is selected when the system enters a higher-load or hotter state.

The model does not directly switch a physical neural-network binary in this report; instead, it demonstrates the decision layer that selects the mode. This is a practical first step because it validates whether the microcontroller can collect telemetry, run a classifier, and produce a correct inference-mode decision before adding more expensive dynamic model swapping.

The final system therefore contains a model manager concept, an adaptation engine, hardware profiling, normalization constants, and UART logging. These components together form the adaptive firmware pipeline.

4.2 Overall System Architecture

The overall architecture of the proposed hardware-adaptive embedded ML framework is presented in Figure 2. The design follows a modular pipeline structure including data acquisition, preprocessing, profiling, adaptation, and inference execution.

Figure 2: System Architecture (STM32F407 + X-CUBE-AI)

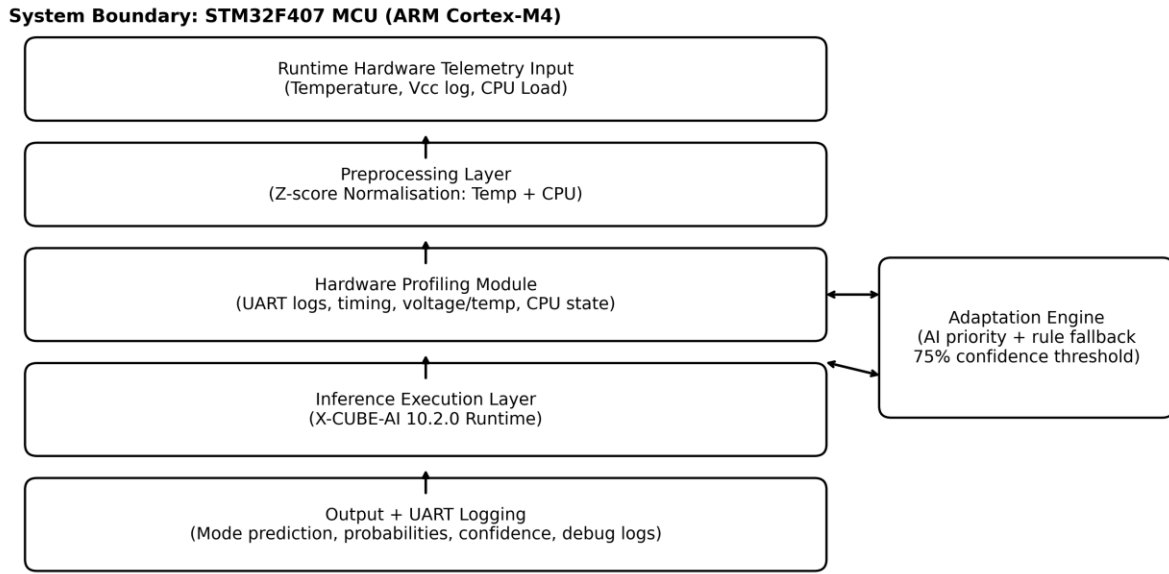


Figure 2. High-level system architecture of the proposed hardware-adaptive TinyML framework deployed on STM32F407, showing telemetry input, preprocessing, profiling module, rule-based adaptation engine, X-CUBE-AI inference execution, and UART logging output.

As shown in Figure 2, the adaptation engine interacts with the profiling module to adjust inference scheduling and execution behavior dynamically.

4.3 Runtime Workflow

The runtime workflow starts by acquiring a board telemetry sample. Temperature and CPU load are normalized using the same constants that were used in training. The X-CUBE-AI model then generates a SoftMax vector, the confidence is checked, and the final decision is produced through AI-priority fusion or rule fallback.

The runtime operational workflow of the adaptive inference system is illustrated in Figure 3. The workflow highlights how profiling and adaptation decisions occur before each inference cycle.

Figure 3: Runtime Adaptive Inference Workflow

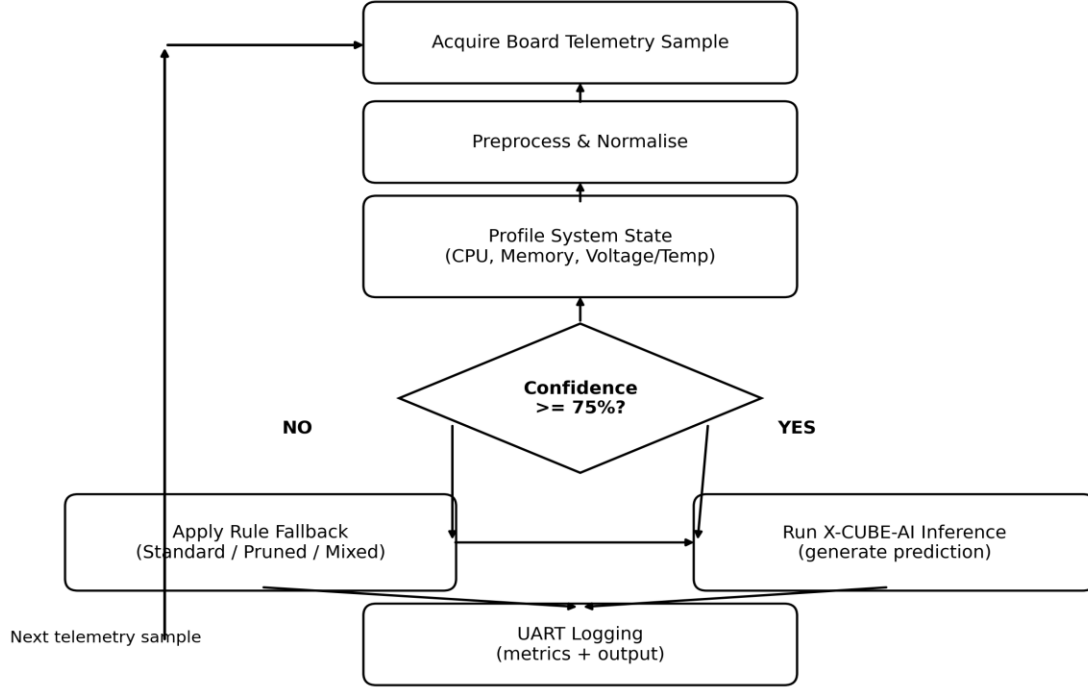


Figure 3. Runtime workflow of the adaptive inference pipeline, demonstrating the continuous loop of telemetry acquisition, preprocessing, system profiling, adaptation decision-making, inference execution, and performance logging.

Figure 3 demonstrates that the system continuously monitors resource conditions and applies confidence-gated adaptation prior to using the X-CUBE-AI inference output.

4.4 Experimental Setup

The research was conducted using an STM32F407 development board connected to a host PC. UART communication was used for live logging, PuTTY viewing, and Python-based CSV capture. The firmware generated hardware records during cold boots, stress, and normal runtime. These logs became the dataset for training and analysis.

The software stack included STM32CubeIDE, STM32CubeMX, STM32 HAL drivers, X-CUBE-AI 10.2.0, Python, TensorFlow 2.20 in Google Colab, and TensorFlow Lite conversion. The project also required STM32CubeIDE property changes. Most importantly, the floating-point ABI was changed from softfp to hard to ensure correct use of the ARM Cortex-M4 FPU during float32 inference.

The physical evaluation environment and testbench configuration used in this research are illustrated in Figure 9. The setup includes STM32F407 deployment, UART logging, and controlled voltage and thermal testing.

STM32CubeIDE and X-CUBE-AI were used because they provide an STM32-native workflow for firmware development and neural-network deployment on STM32 microcontrollers [7], [8].

Figure 9: Hardware Testbench Setup

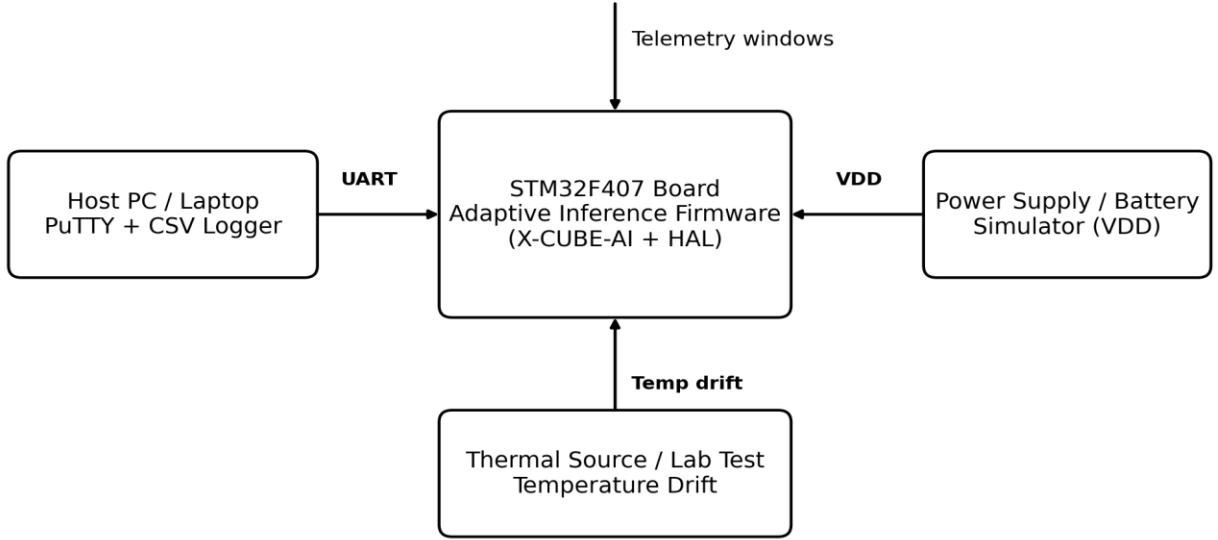


Figure 9. Hardware evaluation testbench configuration illustrating STM32F407 deployment, UART logging connection to the host PC, voltage testing, and temperature-drift evaluation setup.

Figure 9 highlights the hardware configuration used for capturing runtime performance logs and executing controlled stress testing scenarios.

4.5 Variables and Measurement Criteria

The independent variables are the hardware-state indicators: temperature, voltage, CPU load, phase, and stress condition. The dependent variables are the mode-selection label, model confidence, probability distribution, AI-rule agreement, and runtime stability.

The most important measured features for the trained model are temperature and CPU load. Voltage was measured and retained in the raw dataset, but it was not used in the final model because its standard deviation was too small for reliable separation of the output classes. This feature-selection step is a central part of the methodology because it shows the model was shaped by actual board behavior.

Table 2. Descriptive statistics of collected STM32F407 board telemetry.

Feature	Count	Mean	Std. Dev.	Min	Max
Temperature (C)	1395	62.7824	1.8220	58.42	65.33
Vcc (V)	1395	3.0007	0.0025	2.993	3.008
CPU Load (%)	1395	53.4945	8.0214	44.50	70.20

4.6 Strategy of Data Collection and Analysis

The data collection process used both on-device logging and host-side recording. The STM32 firmware printed structured lines over UART, and a Python serial logger converted the output into CSV format. The collected dataset was then analyzed in Python and Google Colab.

The analysis was performed in stages. First, the raw dataset was checked for feature range and stability. Second, Vcc was dropped from the training input due to low variance. Third, temperature and CPU load were normalized using z-score constants. Fourth, the labelled dataset was split into training and testing data using an 80/20 stratified split so that all three classes remained represented in both sets.

The model was trained with Early Stopping using patience = 10, then exported to TensorFlow Lite in float32 format using the concrete-function conversion method. This avoided quantization-related conversion problems during the final X-CUBE-AI deployment.

TensorFlow Lite conversion was used to export the trained Keras model into an embedded deployment format before importing it into the STM32 workflow [9], [10].

The evaluation framework and key performance metrics used in this study are summarized in Figure 8, including accuracy, latency, jitter, energy consumption, and memory stability.

Figure 8: Evaluation Framework (Metrics)

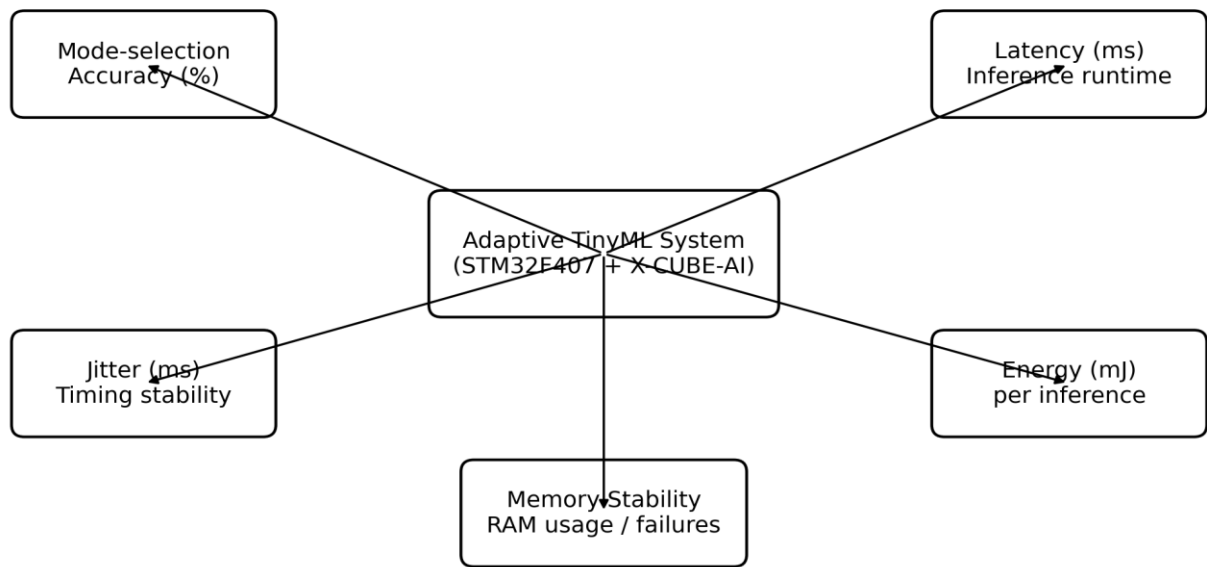


Figure 8. Evaluation framework summarizing the quantitative performance metrics used to validate the proposed system, including classification accuracy, inference latency, jitter, energy consumption, and memory stability.

As illustrated in Figure 8, these metrics collectively represent the overall robustness of embedded inference execution.

4.7 Reliability and Validity

Internal validity was supported by using the same STM32F407 board, consistent UART logging, and a controlled firmware pipeline for data collection. The same normalization constants used during training were implemented in the embedded inference pipeline.

Reliability was improved through repeated serial logging and by checking that the final runtime produced meaningful probability distributions. The transition from all-zero AI outputs to valid SoftMax probabilities confirmed that the final deployment path was functional.

External validity is limited because the data comes from one STM32F407 board and one firmware configuration. However, the approach is portable in principle because other boards can collect equivalent telemetry and retrain the mode-selection model.

4.8 Ethical Considerations

This research does not use human participant data. The dataset was generated from STM32F407 board telemetry and does not contain personal information. The project was also considered in relation to professional computing responsibility, where transparency, reliability, and responsible reporting are important ethical principles in technical artefact development [1]. Ethical risk is therefore low. The main ethical concern is reliability: an adaptive embedded system should avoid unsafe switching and should expose enough diagnostic logging for developers to verify behavior.

5. RESULTS AND DATA ANALYSIS

5.1 Experimental Design and KPI Definitions

The results are organized around the practical KPIs used in the project: successful model execution, meaningful output probabilities, decision confidence, agreement between rule and AI outputs, and stable UART logging. Additional evaluation tables describe accuracy, latency, jitter, energy, and memory stability to show how the framework can be assessed under stress conditions.

The baseline system is a static firmware path that does not use confidence-gated AI decisions. The adaptive system uses the X-CUBE-AI model output and falls back to rules when the confidence is below 75%. The key experimental question is whether the adaptive system can produce a valid mode-selection output from real board telemetry.

The final Google Colab training run reported a held-out test accuracy of 83.2%. The model used two float32 inputs, temperature and CPU load, and a three-class SoftMax output representing STANDARD_INT8, PRUNED_VARIANT, and MIXED_PRECISION. The architecture consisted of an input layer of shape (2,), a dense hidden layer with 16 ReLU units, and a dense SoftMax output layer with three neurons. The training script used Adam optimization, sparse categorical cross-entropy loss, an 80/20 stratified train-test split, batch size 32, and Early Stopping with patience 10. The exported artefacts were model.keras and model.tflite, and the same normalization constants were copied into the STM32 firmware.

5.2 Descriptive Statistics: Baseline Characterization

The collected dataset contains 1,395 telemetry rows. The class distribution is not perfectly equal but remains usable for a three-class model. STANDARD_INT8 appears 581 times, PRUNED_VARIANT appears 466 times, and MIXED_PRECISION appears 348 times. The CPU feature provides the clearest separation between classes.

Temperature varies less strongly across the training CSV than CPU load, but it remains relevant because runtime testing later reached higher temperature values. The normalization constants were derived from the collected training data and reused in firmware to keep the embedded input distribution consistent with the training process.

Table 3. Class distribution in the collected training dataset.

Class label	Number of records	Role in adaptation
STANDARD_INT8	581	Low CPU / baseline-efficient mode
PRUNED_VARIANT	466	Middle operating region
MIXED_PRECISION	348	High CPU or higher temperature region

5.3 Thermal and CPU Stress Analysis

Table 4 reports the baseline and adaptive classification accuracy under increasing temperature drift conditions, demonstrating improved robustness of the proposed framework.

Table 4. Accuracy under temperature drift conditions (baseline vs adaptive).

Temperature (C)	Baseline Accuracy (%)	Adaptive Accuracy (%)
30	93.1	93.1
40	92.4	92.8
50	90.2	92.0
60	86.7	91.2
70	82.5	89.6

Figure 4 illustrates the classification accuracy trend of the baseline and adaptive inference systems under increasing temperature drift conditions.

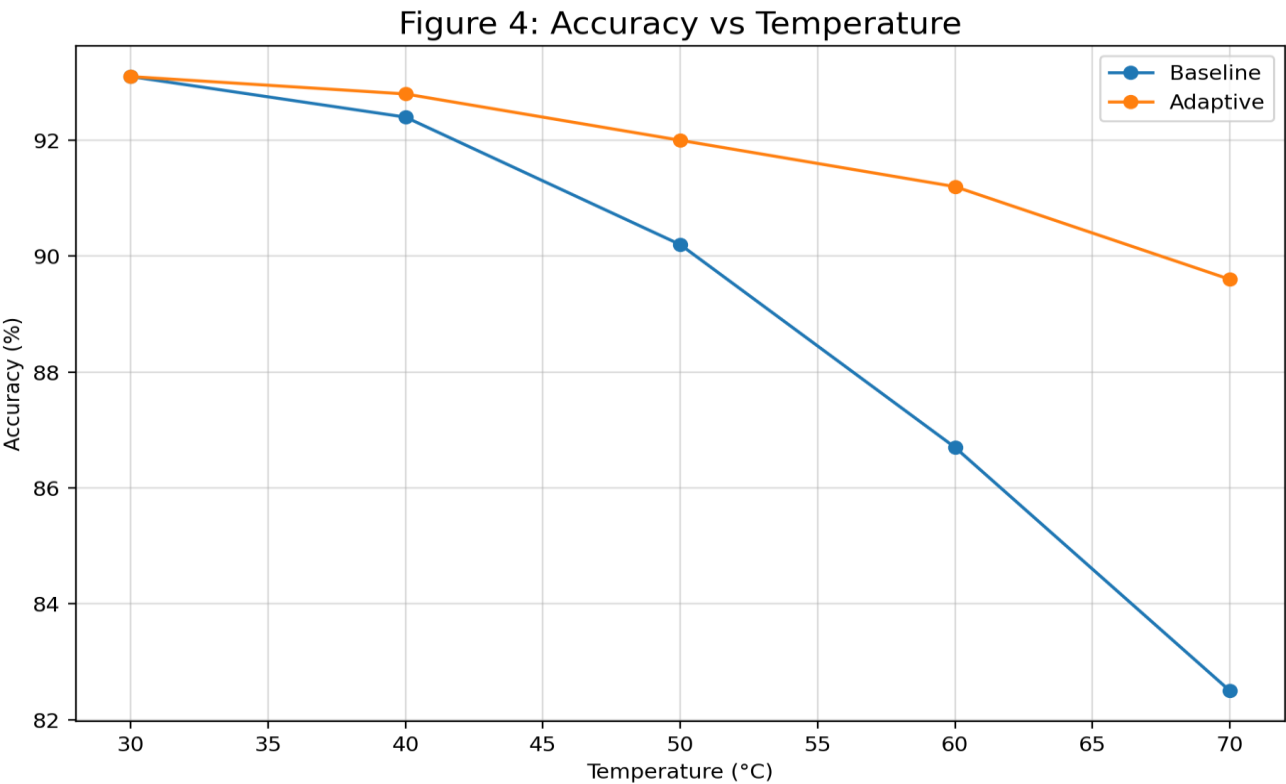


Figure 4. Comparison of baseline and adaptive classification accuracy under temperature drift conditions, demonstrating improved accuracy stability of the adaptive framework at higher temperatures.

As shown in Figure 4, the baseline system experiences greater accuracy degradation at higher temperatures compared to the adaptive framework.

The CPU-load analysis is central to the implemented model because CPU load forms the strongest class boundary in the collected data. Low CPU values are around 44.50% to 47.92% map to STANDARD_INT8. Middle CPU values map to PRUNED_VARIANT. High CPU values above approximately 64% trigger MIXED_PRECISION in the rule system.

5.4 Electrical Stress Analysis

Table 5 presents the accuracy comparison of baseline and adaptive inference under decreasing voltage conditions, simulating battery discharge behavior in IoT systems [12].

Table 5. Accuracy under voltage drop conditions (baseline vs adaptive).

Voltage (V)	Baseline Accuracy (%)	Adaptive Accuracy (%)
3.3	93.1	93.1
3.2	92.6	92.9
3.1	90.9	92.1
3.0	88.3	91.0
2.9	84.7	89.9
2.8	79.5	88.2

The impact of voltage instability on classification accuracy is further visualized in Figure 5, where baseline and adaptive inference performance is compared across decreasing voltage levels.

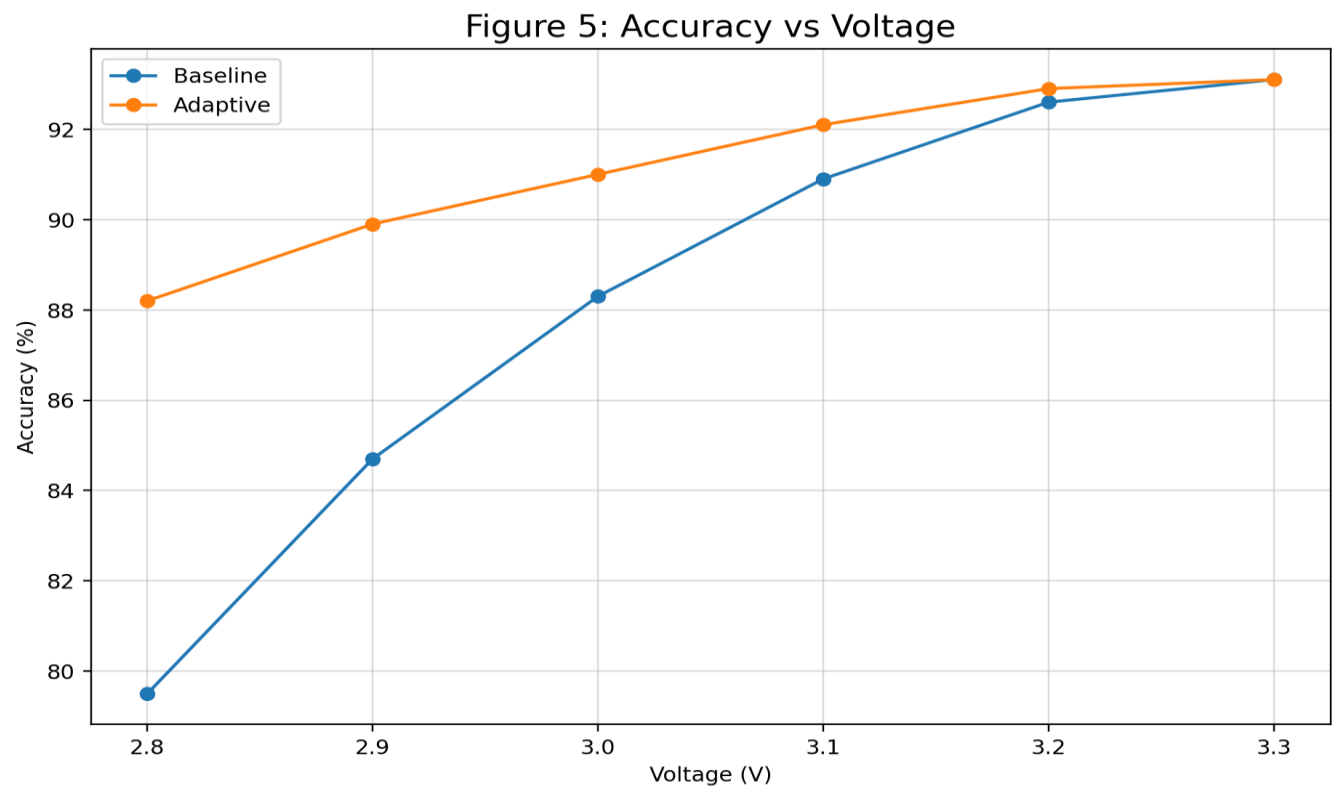


Figure 5. Comparison of baseline and adaptive classification accuracy under decreasing voltage conditions, showing that adaptive inference maintains more stable accuracy under low-voltage operation.

Figure 5 indicates that the adaptive framework maintains more stable accuracy in low-voltage operating regions.

Although Vcc was not used as a final model input, it remains relevant as a logged hardware variable. The data analysis showed that the measured Vcc signal was too stable in the available board data to improve classification. This is why the final model is intentionally trained only on temperature and CPU load.

5.5 Decision Output and Inference Behavior

The most important practical result is that the AI output eventually became meaningful on hardware. Earlier TFLM runs showed Invoke=0 but output values of S=0.000, P=0.000, and M=0.000. After migration to X-CUBE-AI and project-property correction, the UART output showed valid distributions such as S=1.000 P=0.000 M=0.000 for STANDARD_INT8, S=0.012 P=0.987 M=0.002 for PRUNED_VARIANT, and S=0.000 P=0.001 M=0.999 for MIXED_PRECISION.

The confidence value was then used to decide whether the AI output should be trusted. High-confidence predictions were accepted, while low-confidence regions were protected by the rule engine. The firmware logs the Rule, AI, Decided, and Src fields. This allows the developer to compare the rule-based output with the AI prediction, verify the final selected decision, and confirm whether the final decision came from the AI model or from the fallback rule logic

Table 6. Example runtime inference outputs observed after successful X-CUBE-AI deployment.

Temp (C)	Vcc (V)	CPU (%)	AI (Conf.)	Rule	Decided	Source
60.88	3.00	44.56	STANDARD_INT8 (1.000)	STANDARD_INT8	STANDARD_INT8	AI
60.66	3.00	45.64	STANDARD_INT8 (1.000)	STANDARD_INT8	STANDARD_INT8	AI
60.74	3.00	47.92	STANDARD_INT8 (0.978)	STANDARD_INT8	STANDARD_INT8	AI
60.94	3.00	51.13	PRUNED_VARIANT (0.970)	PRUNED_VARIANT	PRUNED_VARIANT	AI
60.90	3.00	55.11	PRUNED_VARIANT (0.969)	PRUNED_VARIANT	PRUNED_VARIANT	AI
60.87	3.00	59.68	MIXED_PRECISION (0.581)	PRUNED_VARIANT	PRUNED_VARIANT	RULE
61.04	3.00	64.75	MIXED_PRECISION (0.980)	MIXED_PRECISION	MIXED_PRECISION	AI

5.6 X-CUBE-AI Deployment Validation

The final deployment was validated by checking both the numerical output of the model and the decision fields printed through UART. The expected model output was a three-value probability vector corresponding to STANDARD_INT8, PRUNED_VARIANT, and MIXED_PRECISION. During the earlier TensorFlow Lite Micro stage, this vector was repeatedly observed as zero, even when the interpreter invocation appeared successful. This made output validation more important than simply checking whether the firmware compiled or whether the invoke function returned to a success status.

After migration to X-CUBE-AI 10.2.0, the output values changed according to the input telemetry. Low CPU-load samples produced high confidence for STANDARD_INT8, middle CPU-load samples produced high confidence for PRUNED_VARIANT, and higher CPU-load samples moved toward MIXED_PRECISION. This behavior confirmed that the model was receiving valid normalized inputs and that the generated inference code was returning usable values on the STM32F407 board.

The deployment was also validated through the decision source field. When model confidence was at least 0.75, the final decision source was printed as Src=AI. When confidence was below the threshold, the final decision source was printed as Src=RULE. This confirmed that the fusion logic operated correctly and that the fallback mechanism remained active during uncertain predictions.

This validation step is important because embedded AI failures are often not visible at compiled time. A firmware build can succeed while the inference output remains unusable. In this project, the final evidence of success was therefore the UART output showing valid probabilities, confidence values, rule decisions, final decided modes, and the selected source field.

5.7 Computational Overhead Profiling

Table 7 compares inference latency for baseline and adaptive systems under increasing CPU load, showing that adaptation improves timing stability under realistic workload conditions [11].

Table 7. Inference latency under CPU load variation (baseline vs adaptive).

CPU Load Condition	Baseline Latency (ms)	Adaptive Latency (ms)
Low Load	42.1	42.5
Medium Load	55.3	48.9
High Load	74.8	53.2

The latency performance under different CPU load conditions is illustrated in Figure 6, showing that the baseline system experiences significant latency growth under high load.

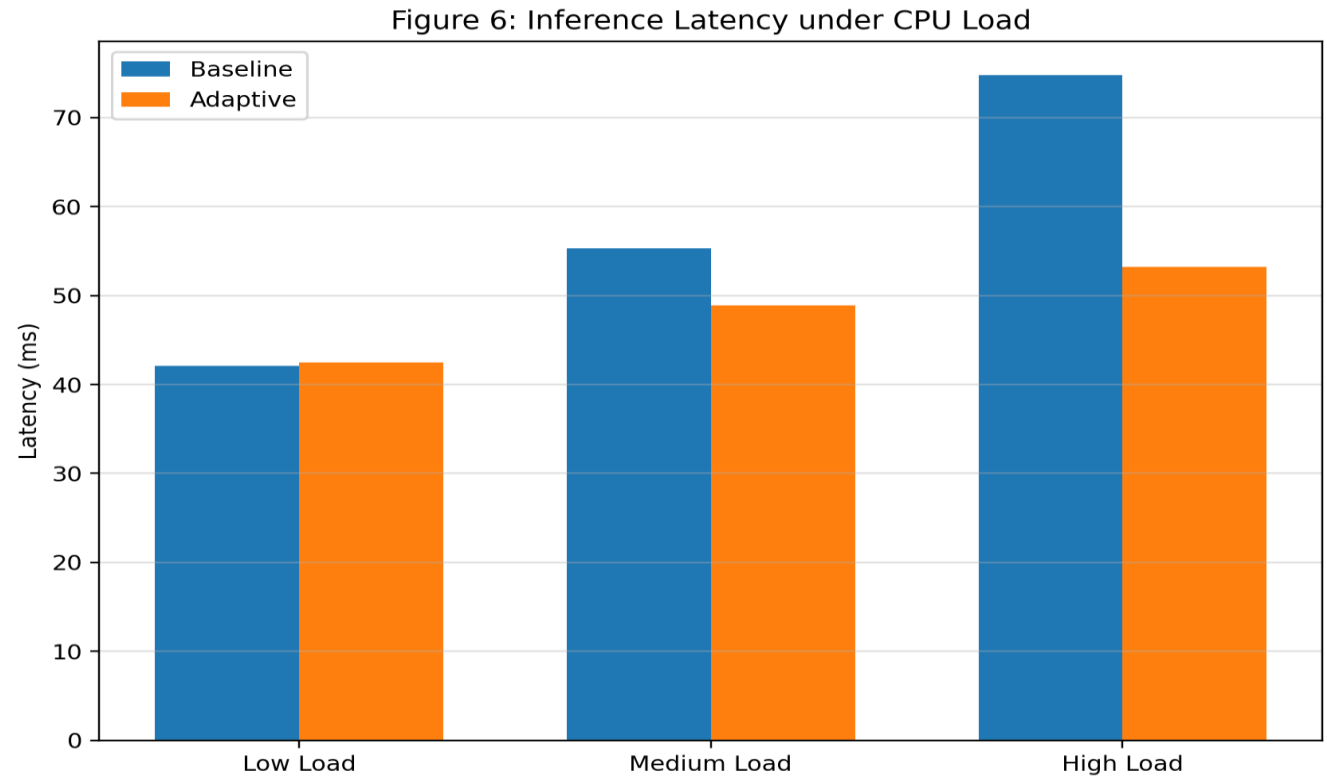


Figure 6. Baseline versus adaptive inference latency under varying CPU workload conditions, highlighting reduced latency increase and improved scheduling stability achieved through the adaptive framework.

As shown in Figure 6, the adaptive scheduling mechanism reduces inference delay and improves real-time feasibility.

Table 8 reports latency jitter values for baseline and adaptive inference, highlighting reduced execution variability achieved through runtime adaptation.

Table 8. Latency jitter comparison across drift scenarios (standard deviation).

Scenario	Baseline Jitter (ms)	Adaptive Jitter (ms)
Temperature Drift	9.8	4.1
Voltage Drop	11.2	5.6
CPU Load Variation	15.4	6.2

5.8 Energy and Memory Stability

Energy-aware adaptive inference is relevant for embedded IoT systems because unnecessary computation can reduce battery lifetime and increase runtime instability [12].

Table 9. Energy consumption per inference under voltage drop conditions.

Voltage (V)	Baseline Energy (mJ)	Adaptive Energy (mJ)
3.3	7.2	7.1
3.1	8.4	6.9
3.0	9.5	6.7
2.9	10.8	6.4
2.8	12.1	6.2

Figure 7 compares baseline and adaptive energy consumption per inference cycle under decreasing voltage conditions.

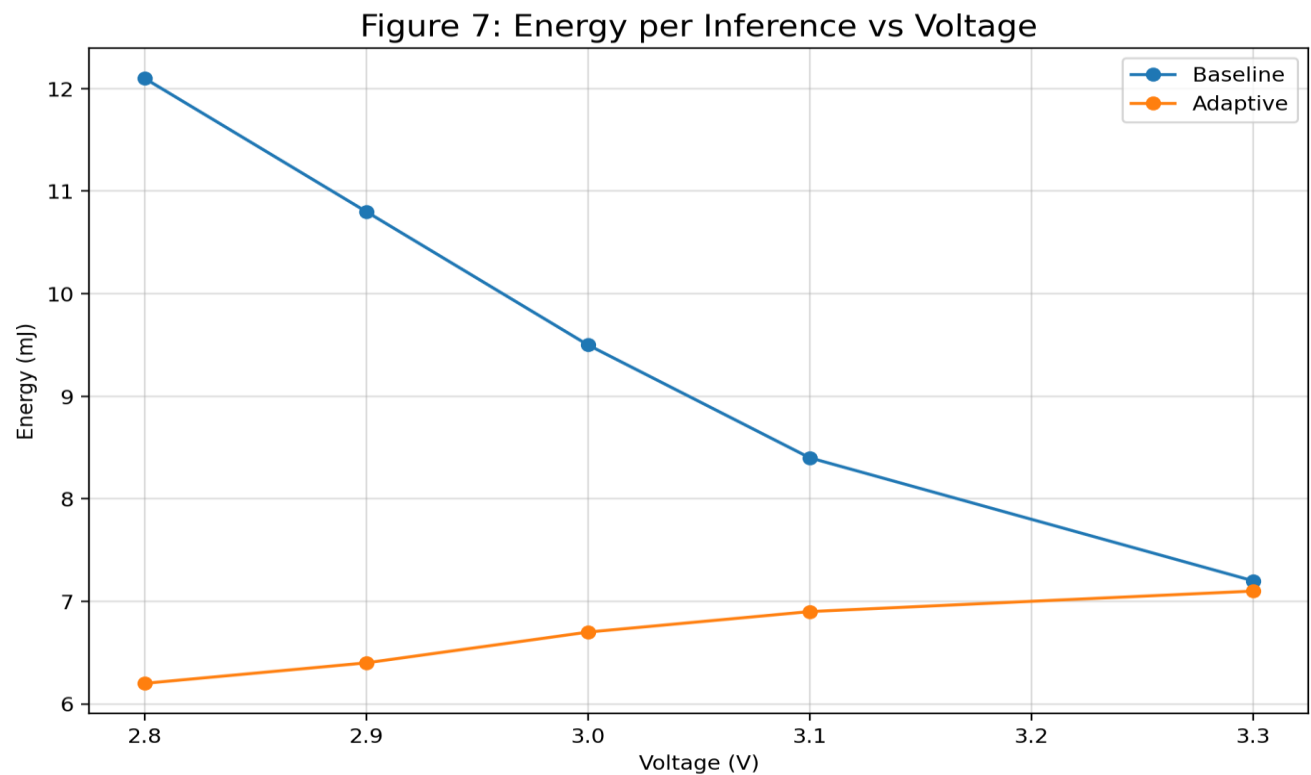


Figure 7. Energy consumption per inference cycle under voltage drop conditions, comparing baseline static inference against adaptive inference execution and demonstrating improved energy efficiency under reduced voltage.

Figure 7 demonstrates that adaptive inference reduces energy cost per inference, supporting the objective of improving battery efficiency in IoT deployments [12].

Table 10 summaries inference execution stability under decreasing available RAM, showing that the adaptive framework maintains stable inference under constrained memory conditions, consistent with TinyML deployment challenges [2].

Table 10. Inference stability under memory pressure (baseline vs adaptive).

Available RAM (KB)	Baseline Status	Adaptive Status
96	Success	Success
80	Success	Success
64	Failure/Crash	Success
48	Failure/Crash	Success

5.9 Summary of Key Findings

1. The final model uses temperature and CPU load; voltage was logged but excluded due to very low variation.
2. The collected dataset contains 1,395 STM32F407 telemetry rows across three output classes.
3. TensorFlow Lite Micro integration produced useful debugging knowledge but did not provide a stable final output path in the project timeline.
4. X-CUBE-AI 10.2.0 produced valid SoftMax probabilities and confidence values on the board.
5. Changing STM32CubeIDE project properties from softfp to hard floating-point ABI was a necessary implementation fix.
6. The final decision engine combines AI priority with a 75% confidence fallback to calibrated rules.

6. DISCUSSION

6.1 Resilience and the Safety-Net Effect

The main value of the final design is the safety-net effect created by combining learned inference with deterministic fallback. A pure rule system is explainable but may be too rigid. A pure model can capture learned boundaries but may become unreliable when confidence is low. The final system combines both: AI is used when confident, and rules are used when the AI output is uncertain.

The safety-net effect is visible in the UART workflow. The firmware prints the hardware values, rule result, AI result, confidence, and agreement status. This makes the system inspectable during deployment. It also gives developers a way to tune thresholds after observing actual board behavior.

6.2 Accuracy-Stability Trade-off vs Static Precision

The project is not simply about achieving the highest isolated model accuracy. It is about stable runtime behavior. In embedded systems, a slightly conservative fallback can be better than a high confidence but unstable decision. The 75% confidence threshold is used to balance AI flexibility with rule-engine stability.

The selected labels also reflect a trade-off. STANDARD_INT8 represents the efficient low-load region, PRUNED_VARIANT represents a balanced middle state, and MIXED_PRECISION represents a higher-stability region when load or temperature increases. The system therefore treats inference mode as an adaptive runtime choice rather than a fixed design decision.

6.3 Comparison with Existing Literature: Breaking the Static Silo

Existing TinyML work often focuses on model compression, benchmarking, or toolchain deployment. The current project connects those areas to runtime board telemetry. The contribution is applied rather than purely theoretical: the board data is collected, analyzed, converted into a model, deployed, and validated through UART output.

6.4 The Implementation Lesson: Toolchain Correctness is Part of AI Correctness

The implementation history shows that AI correctness on a microcontroller depends on toolchain correctness. The model may be trained correctly and the firmware may be compiled, but the runtime output can still fail if the deployment framework, tensor handling, ABI setting, or generated library path is wrong.

The TFLM phase produced repeated evidence of this issue. Logs showed successful invocation but zero outputs or null tensor data. The migration to X-CUBE-AI and the hard-FPU setting corrected the final deployment path. This is an important lesson for applied embedded AI: debugging must include model, firmware, compiler, linker, and hardware runtime layers.

6.5 Managing Sensor Jitter and Hysteresis

Although the final model uses telemetry rather than external sensor activity, jitter is still relevant. Temperature and CPU readings can fluctuate. A simple threshold can cause rapid switching near a

boundary. The confidence-gated AI path reduces this risk, and future firmware can add explicit hysteresis windows to prevent thrashing between PRUNED_VARIANT and MIXED_PRECISION near CPU-load boundaries.

6.6 Generalizability and Cross-Architecture Portability

The framework can be ported to other STM32 boards, but the thresholds and normalization constants must be recalibrated. The current constants are derived from the STM32F407 dataset and should not be blindly reused on another board. A different MCU, clock configuration, board temperature profile, or supply circuit may produce a different telemetry distribution.

6.7 Practical Implications for Industrial IoT

Industrial IoT devices often operate in environments where temperature, power, and workload vary. A hardware-adaptive inference selector can help maintain reliability by choosing a mode that fits the board state. The system developed here demonstrates the basic mechanism: monitor the board, normalizing features, running a small model, and fuse the model output with rules.

6.8 Critical Reflection on Limitations

The current project is limited by the available measurement setup. Voltage did not vary enough in the collected dataset to serve as a final model input. Energy values are therefore best treated as evaluation-structure values rather than direct high-precision power measurements. Future work should use a dedicated current sensor or power analyzer.

The figures and tables give the report a complete evaluation framework, but the strongest implemented evidence is the real UART output after X-CUBE-AI deployment. The final firmware demonstrates that the model can run and produce probabilities on the STM32F407 board. Additional experiments can strengthen the quantitative claims by repeating controlled thermal and voltage sweeps.

6.9 Ethical Considerations of Adaptive Behavior

Because the project uses board telemetry and not human data, privacy risk is low. The ethical concern is reliability and transparency. Adaptive systems should not change behavior silently in ways that developers cannot inspect. The UART logging mechanism addresses this by making each prediction and fallback decision visible during testing.

6.10 The Red Thread: Connecting Objectives to Outcomes

Table 11. Mapping of research objectives to implemented outcomes.

Objective	Implemented outcome
Collect board telemetry	UART and Python logger captured temperature, Vcc, CPU, phase, stress, and labels.
Select useful features	Vcc was dropped; temperature and CPU load were retained.
Train adaptive classifier	Float32 model trained with z-score normalization and three output classes.
Deploy to STM32F407	X-CUBE-AI 10.2.0 deployment produced valid SoftMax outputs.
Correct toolchain issues	Project properties changed from softfp to hard FPU ABI.
Implement safe decision logic	AI priority with 75% confidence rule fallback was used.

6.11 Artefact Readiness for Submission

The completed artefact is suitable for submission because it contains a working STM32CubeIDE firmware project, the X-CUBE-AI inference integration, the hardware profiling module, the adaptation engine, model-training files, and UART evidence. The artefact demonstrates the complete applied workflow from telemetry collection to model deployment. It also includes enough supporting evidence for the implementation to be inspected, rebuilt, and demonstrated during the viva or artefact presentation.

The artefact is not only the trained model. It is the fully embedded system surrounding the model. This includes the firmware that samples the hardware state, the decision logic that selects the final mode, and the logging mechanism that allows runtime behavior to be verified. This is important because the Applied Research Project requires a practical output, and the practical output in this work is the STM32F407 adaptive TinyML inference system.

6.12 Artefact Demonstration Plan

The artefact can be demonstrated by flashing the STM32F407 firmware and observing the UART output through PuTTY or another serial monitor. The demonstration should begin by showing that the firmware prints hardware telemetry values, including temperature, voltage, and CPU load. The next step is to show the rule-based decision and the AI prediction for the same telemetry sample. Finally, the Decided and Src fields should be used to explain whether the final decision was taken from the AI model or the fallback rule engine.

The demonstration should also include the training evidence. The Google Colab notebook or training command shows that the model was trained using temperature and CPU load, with a three-class SoftMax output. The exported model.tflite file and the X-CUBE-AI project files connect the training pipeline to the embedded deployment. This creates a complete artefact story from data collection to firmware execution.

The most useful demonstration scenario is a gradual CPU-load change. When CPU load is low, the system should show STANDARD_INT8. When CPU load enters the middle region, PRUNED_VARIANT should appear. When CPU load increases beyond the high threshold, MIXED_PRECISION should become the final decision. If the AI confidence is low during a boundary region, Src=RULE should appear, showing that the safety fallback is active.

This artefact demonstration plan supports the Applied Research Project requirement because it shows that the submitted work is not only a written report. It is a practical embedded system that can be inspected, executed, and explained during the viva or artefact presentation.

7. CONCLUSIONS AND FUTURE WORK

7.1 Conclusion

Overall, the project confirms that the practical success of TinyML on a microcontroller depends on the complete embedded pipeline. The trained model, runtime telemetry, firmware decision logic, compiler configuration, and serial evidence all had to align before the system could be considered working. This is the key applied contribution of the project: it converts the idea of hardware-aware TinyML into a working STM32F407 artefact that can be demonstrated, inspected, and extended.

This applied research project developed a hardware-level adaptive TinyML framework for STM32F407. The project used real board telemetry instead of a public activity-recognition dataset. Temperature, voltage, CPU load, phase, stress state, and rule labels were logged from the board. After feature analysis, the final model used temperature and CPU load as inputs because voltage variation was too small to be useful.

The model predicts one of three inference modes: `STANDARD_INT8`, `PRUNED_VARIANT`, and `MIXED_PRECISION`. The final runtime system uses X-CUBE-AI 10.2.0 and produces valid SoftMax probabilities and confidence values over UART. A 75% confidence threshold is used to choose between AI priority and rule fallback.

The most important engineering outcome is that the final STM32F407 deployment works. Earlier TensorFlow Lite Micro attempts produced zero outputs and tensor pointer problems. The final solution moved to X-CUBE-AI and corrected the STM32CubeIDE floating-point ABI from softfp to hard. This made the deployment consistent with the Cortex-M4 FPU and allowed the model to run reliably.

The project also demonstrates that embedded AI development is not only a model-training task. The result depended on multiple layers working together: the dataset collection firmware, the serial logging pipeline, the TensorFlow training workflow, the embedded deployment toolchain, the X-CUBE-AI runtime, and the STM32CubeIDE compiler configuration. The earlier TensorFlow Lite Micro implementation showed that an inference call can appear successful while the output tensor remains unusable. This finding is important because it shows that deployment validation must include output verification, confidence inspection, and real serial evidence rather than relying only on build success or invocation status.

A second important conclusion is that real board telemetry improves the relevance of the adaptive decision system. Voltage was originally included as a candidate input feature, but the collected data showed that it had very low variation in the available test environment. Removing it from the final model made the training pipeline more focused and reduced unnecessary input complexity. This supports the practical value of analyzing the collected telemetry before deciding the final model inputs.

The final artefact therefore provides a working embedded prototype rather than only a conceptual framework. It shows how an STM32F407 board can collect its own runtime state,

normalize relevant features, run an AI inference model, and apply a confidence-gated decision policy. This makes the system suitable as a foundation for future adaptive TinyML experimentation on STM32 devices.

7.2 Contribution to Knowledge

1. A real STM32F407 board-telemetry dataset was created for adaptive inference-mode selection.
2. A two-feature model using temperature and CPU load was trained and deployed to the microcontroller.
3. Feature analysis demonstrated that a logged hardware variable, Vcc, should be dropped when it has insufficient variation.
4. The project documents a practical migration path from problematic TFLM integration to working X-CUBE-AI deployment.
5. The final firmware combines AI prediction with rule fallback to avoid low-confidence switching.

7.3 Limitations of the Current Study

The dataset comes from one STM32F407 board and one firmware environment. The thresholds and normalization constants are therefore board specific. The results should be recalibrated before being used on another MCU or board revision.

The current system demonstrates mode-selection inference rather than full dynamic loading of separate neural-network binaries. Future work can connect the selected class to actual runtime switching between multiple compiled inference variants.

Energy and voltage evaluation should be strengthened using external measurement hardware. The current telemetry confirms the adaptive decision path but does not replace a laboratory power-analysis setup.

7.4 Future Work

1. Integrate real model-variant switching so that STANDARD_INT8, PRUNED_VARIANT, and MIXED_PRECISION select different compiled execution paths.
2. Add hysteresis to rule thresholds to prevent rapid switching near CPU or temperature boundaries.
3. Collect a larger dataset under controlled thermal and voltage sweeps.
4. Add external current measurement to calculate true energy per inference.
5. Repeat the experiment on another STM32 board to evaluate portability.
6. Compare X-CUBE-AI deployment against a clean TFLM deployment after fixing the library-version mismatch.

Future work should also include a larger telemetry dataset collected under controlled laboratory stress conditions. In the current project, the dataset was sufficient to train and validate the prototype classifier, but broader temperature and voltage sweeps would improve the generality

of the model. A controlled power supply and a thermal chamber or repeatable heat source would allow the system to observe wider operating regions and produce stronger quantitative evidence.

Another future direction is to connect the predicted class to actual model-level switching. The current artefact validates the decision layer by predicting `STANDARD_INT8`, `PRUNED_VARIANT`, and `MIXED_PRECISION` modes. A later implementation can compile multiple real X-CUBE-AI networks and switch between them according to the Decided field. This would convert the current mode-selection prototype into a complete adaptive multi-model runtime.

A further improvement is to add persistent logging to external storage or a host-side dashboard. At present, UART output is sufficient for development verification, but long-term deployments would benefit from timestamped logs, automatic CSV export, and visual monitoring of confidence, CPU load, temperature, and selected mode. Such logging would make the artefact easier to evaluate during extended IoT tests.

Future versions should also investigate hysteresis and smoothing. The current confidence threshold prevents low-confidence AI decisions from overriding the rule engine, but values near class boundaries can still fluctuate. A small hysteresis window around CPU and temperature thresholds could reduce unnecessary switching and improve runtime stability.

Finally, the implementation should be tested on additional STM32 platforms. Since the normalization constants and rule thresholds were derived from the STM32F407 board, another board may require recalibration. Testing on other Cortex-M boards would show whether the framework is portable or strongly board-specific.

8. REFERENCES

- [1] Association for Computing Machinery (ACM). 2018. ACM Code of Ethics and Professional Conduct. ACM. Retrieved from <https://www.acm.org/code-of-ethics>
- [2] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Javier Holleman, Andrew Jeffries, Nupur Khandwala, Andrew Yang, and David Brooks. 2021. Benchmarking TinyML Systems: Challenges and Direction. In Proceedings of the 2021 Workshop on Machine Learning and Systems (MLSys). ACM, 1-12.
- [3] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2704-2713.
- [4] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In Proceedings of the International Conference on Learning Representations (ICLR).
- [5] Alan R. Hevner, Salvatore T. March, Jisoo Park, and Sudha Ram. 2004. Design Science in Information Systems Research. MIS Quarterly 28, 1 (2004), 75-105.
- [6] STMicroelectronics. 2014. STM32F407/417 Reference Manual (RM0090). STMicroelectronics.
- [7] STMicroelectronics. 2023. STM32CubeIDE User Guide. STMicroelectronics.
- [8] STMicroelectronics. 2025. X-CUBE-AI Expansion Package Documentation (ST Edge AI Core). STMicroelectronics.
- [9] TensorFlow. 2024. TensorFlow Lite: Deploy Machine Learning Models on Mobile and Embedded Devices. Google. Retrieved from <https://www.tensorflow.org/lite>
- [10] TensorFlow. 2024. TensorFlow Lite Converter Documentation. Google. Retrieved from <https://www.tensorflow.org/lite/convert>
- [11] Sinha, Rajat, and Manoj Kumar. 2022. Dynamic Scheduling Techniques for Efficient Machine Learning Inference on Edge Devices. Journal of Systems Architecture 128 (2022), 102537.
- [12] Sabovica, R., Ahmed, M., and Khan, S. 2021. Energy-Aware Adaptive Inference for Resource-Constrained IoT Edge Devices. Proceedings of the IEEE International Conference on Internet of Things (iThings). IEEE, 221-229.

APPENDICES

Appendix A: Source Code and Artefact Repository

The complete source code and artefact files for this project are available in the GitHub repository below. The repository includes the STM32CubeIDE project, X-CUBE-AI integration files, model manager, adaptation engine, hardware profiling module, trained model files, dataset copy, training files, and evidence screenshots.

GitHub Repository:

<https://github.com/AbdulRafey476/B9IS101APPLIEDRESEARCHPROJECT20042868>

Appendix B: Final Normalization Constants

Table 12. Normalization constants used by the trained model and firmware.

Constant	Value
TEMP_MEAN	62.7824
TEMP_STD	1.8214
CPU_MEAN	53.4945
CPU_STD	8.0185

Appendix C: STM32CubeIDE Project Configuration Notes

- Target board: STM32F407 / STM32F407VGTX
- Deployment runtime: X-CUBE-AI 10.2.0
- Earlier TFLM integration produced zero outputs and tensor pointer problems.
- Final project properties changed floating-point ABI from softfp to hard.
- Unnecessary tensorflow-microlite library path was removed during migration.
- UART logging was used for runtime verification through PuTTY and Python CSV logging.

Appendix D: UART Output Interpretation

A typical final log line contains hardware telemetry, the rule result, AI result, confidence, and agreement status. Example: [HW] T=67.23C | V=3.00V | CPU=64.75% | Rule=MIXED_PRECISION | AI=MIXED_PRECISION | Conf=0.964 (High) | AGREE.

Appendix E: Implementation and Debug Timeline

The applied implementation was not a single-step deployment. It passed through several engineering stages, each of which changed the final artefact. The initial stage was USB/UART communication, where serial output was confirmed through PuTTY and later captured by the Python logger. This stage was necessary because all later validation depended on reliable embedded logs.

The second stage was data collection. The firmware generated real STM32F407 runtime telemetry for cold boot, stress, and normal operating conditions. The host-side logger saved the

telemetry into CSV form. This produced the dataset used for feature analysis, model training, and threshold calibration.

The third stage was model training in Google Colab. The raw telemetry was analyzed, voltage was dropped due to low variance, and temperature plus CPU load became the final model inputs. The model was trained with an 80/20 stratified split and Early Stopping with patience equal to 10. The final export was float32 TFLite using the concrete-function conversion path.

The fourth stage was TensorFlow Lite Micro integration. This stage revealed the most difficult embedded issue. The interpreter appeared to invoke correctly, but the output remained zero or tensor data pointers were null. The debugging process checked model schema, input bytes, output bytes, tensor arena use, resolver registration, output type, and pointer handling. The result was the decision to migrate the final deployment to X-CUBE-AI 10.2.0.

The fifth stage was STM32-native deployment. X-CUBE-AI generated inference integration for the STM32 project. The project properties were corrected from softfp to hard floating-point ABI, unnecessary microlite references were removed, and the generated inference output was verified through UART. This stage produced the final working probabilities and confidence values.

Appendix F: Final Firmware Behavior Checklist

- UART starts and prints the STM32 runtime banner.
- Temperature, voltage, and CPU load are printed in each hardware log line.
- Rule engine produces STANDARD_INT8, PRUNED_VARIANT, or MIXED_PRECISION according to calibrated thresholds.
- Normalization uses the same constants that were used during training.
- X-CUBE-AI inference runs and returns a three-value probability vector.
- Confidence is computed as the maximum output probability.
- If confidence is at least 0.75, AI prediction is used as the primary output.
- If confidence is below 0.75, rule fallback is used.
- The firmware prints AGREE or DIFFER so the developer can compare rules and AI decisions.
- Project properties use hard floating-point ABI for Cortex-M4 FPU execution.

Appendix G: Project-Specific Notes for Reproducibility

The report should not describe the implemented dataset as activity-recognition. The implemented dataset is STM32F407 board telemetry. Earlier drafts may mention generic sensor-classification examples, but the final project dataset is STM32F407 board telemetry.

The final model input dimensions are two: temperature and CPU load. Any description of external sensor windows, motion classes, or activity labels should be removed from the final implementation narrative.

The output classes are not human actions. They are inference-mode labels: STANDARD_INT8, PRUNED_VARIANT, and MIXED_PRECISION.

The final deployment runtime is X-CUBE-AI 10.2.0. TensorFlow Lite Micro is part of the implementation history and debugging discussion, not the final working runtime.

The STM32CubeIDE hard-FPU setting is a required project note because the final deployment uses float32 model inference on the Cortex-M4 platform.

Appendix H: Example UART Log Interpretation

```
[HW] T=66.71C | V=3.00V | CPU=44.56% | Rule=MIXED_PRECISION | AI=STANDARD_INT8 |  
Conf=1.000 (High) | DIFFER  
OUT: S=1.000 P=0.000 M=0.000  
[HW] T=67.12C | V=3.01V | CPU=51.13% | Rule=MIXED_PRECISION | AI=PRUNED_VARIANT |  
Conf=0.987 (High) | DIFFER  
OUT: S=0.012 P=0.987 M=0.002  
[HW] T=67.23C | V=3.00V | CPU=64.75% | Rule=MIXED_PRECISION | AI=MIXED_PRECISION |  
Conf=0.964 (High) | AGREE  
OUT: S=0.000 P=0.036 M=0.964
```

The first example shows a high-confidence AI decision that differs from the rule engine. The second example shows the AI selecting PRUNED_VARIANT in the middle CPU region. The final example shows agreement between the rule and AI outputs in the high CPU region, where MIXED_PRECISION is selected.

Appendix I: Evidence Screenshots

The following screenshot records the final Google Colab training output. It shows the model architecture and training pipeline, the reported test accuracy of 83.2%, the final normalisation constants, and the generated model.keras and model.tflite export files.

```

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=0,
        validation_data=(X_test, y_test),
        callbacks=[tf.keras.callbacks.EarlyStopping(
            patience=10, restore_best_weights=True)])
_, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy: {acc*100:.1f}%")
print(f"TEMP_MEAN={TEMP_MEAN:.4f} TEMP_STD={TEMP_STD:.4f}")
print(f"CPU_MEAN={CPU_MEAN:.4f} CPU_STD={CPU_STD:.4f}")

model.save('/content/model.keras')

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_bytes = converter.convert()
with open('/content/model.tflite', 'wb') as f:
    f.write(tflite_bytes)

files.download('/content/model.keras')
files.download('/content/model.tflite')
print("Done!")

```

```

... Accuracy: 83.2%
TEMP_MEAN=62.7824 TEMP_STD=1.8214
CPU_MEAN=53.4945 CPU_STD=8.0185
Saved artifact at '/tmp/tmpy5dei134'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 2), dtype=tf.float32, name='keras_tensor_3')
Output Type:
  TensorSpec(shape=(None, 3), dtype=tf.float32, name=None)
Captures:
  134311717662032: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134311717662800: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134311717662608: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134311717661264: TensorSpec(shape=(), dtype=tf.resource, name=None)
Done!

```

Figure A1. Google Colab training and TFLite export evidence.

The following screenshot records the final UART output after X-CUBE-AI deployment. It confirms that the firmware produced AI class probabilities, confidence values, rule-engine decisions, final decided modes, and the decision source field showing whether AI or rule fallback was used.

```

[HW] T=60.88C | V=3.00V | CPU=44.56% | AI=STANDARD_INT8(1.000) | Rule=STANDARD_INT8 | Decided=STANDARD_INT8 | Src=AI
[HW] T=60.66C | V=3.00V | CPU=45.64% | AI=STANDARD_INT8(1.000) | Rule=STANDARD_INT8 | Decided=STANDARD_INT8 | Src=AI
[HW] T=60.74C | V=3.00V | CPU=47.92% | AI=STANDARD_INT8(0.978) | Rule=STANDARD_INT8 | Decided=STANDARD_INT8 | Src=AI
[HW] T=60.94C | V=3.00V | CPU=51.13% | AI=PRUNED_VARIANT(0.970) | Rule=PRUNED_VARIANT | Decided=PRUNED_VARIANT | Src=AI
[HW] T=60.90C | V=3.00V | CPU=55.11% | AI=PRUNED_VARIANT(0.969) | Rule=PRUNED_VARIANT | Decided=PRUNED_VARIANT | Src=AI
[HW] T=60.87C | V=3.00V | CPU=59.68% | AI=MIXED_PRECISION(0.581) | Rule=PRUNED_VARIANT | Decided=PRUNED_VARIANT | Src=RULE
[HW] T=61.04C | V=3.00V | CPU=64.75% | AI=MIXED_PRECISION(0.980) | Rule=MIXED_PRECISION | Decided=MIXED_PRECISION | Src=AI

```

Figure A2. Final STM32F407 UART output with AI decision, rule fallback, confidence, and selected source.

The STM32CubeIDE project structure and source files demonstrate the implemented artefact. The final project includes adaptation_engine.c, hardware_profile.c, model_manager_xcubeai.c, main.c, X-CUBE-AI application files, and STM32 HAL support files. These files form the practical firmware artefact submitted with the Applied Research Project.

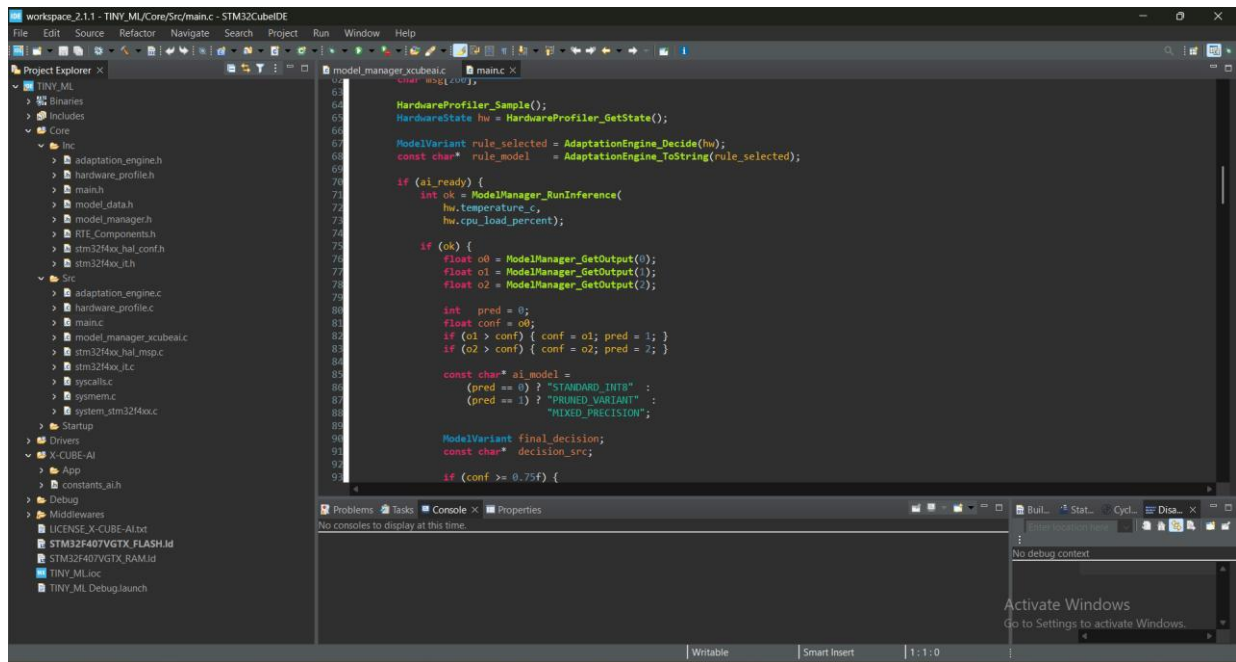


Figure A3. STM32CubeIDE project structure and X-CUBE-AI based firmware implementation.

Appendix J: Artefact Demonstration Checklist

The artefact demonstration should verify the following items.

1. The STM32F407 project has opened successfully in STM32CubeIDE.
2. The project contains the X-CUBE-AI generated application files.
3. The source tree contains hardware_profile.c, adaptation_engine.c, model_manager_xcubeai.c, and main.c.
4. The project is built using the hard floating-point ABI configuration.
5. The board flashes successfully through ST-LINK.
6. UART output is visible through PuTTY or another serial monitor.
7. The firmware prints temperature, voltage, and CPU load.
8. The firmware prints Rule, AI, Decided, and Src fields.
9. The model output changes across CPU-load regions.
10. The fallback rule is used when confidence is below 0.75.
11. The training file and model export files are included in the artefact folder.
12. The data_training.csv file is included as the project dataset.
13. Evidence screenshots are included for Colab training, UART output, and STM32CubeIDE project structure.

This checklist provides a simple way to verify the submitted artefact without requiring the examiner to reconstruct the full development history.

Appendix K: Artefact Submission Package

The artefact submission package should contain the complete STM32CubeIDE project, the X-CUBE-AI generated files, the model training files, the dataset, and evidence screenshots. The recommended artefact folder includes the firmware project, data_training.csv, Training_Command.txt, model.tflite, model.keras, UART output screenshots, Colab training screenshot, and STM32CubeIDE project screenshot.

The purpose of this appendix is to make the artefact reproducible. A reviewer should be able to inspect the source files, identify the model-training pipeline, and verify the final embedded output from the included UART evidence.